

Bapuji Educational Association (Regd.)

Bapuji Institute of Engineering and Technology

An Autonomous Institute Affiliated to Visvesvaraya Technological University, Belagavi,
Karnataka.

Department of Artificial Intelligence & Machine Learning

Analysis and Design of Algorithms (ADA)

Module 5 – Limitations of Algorithmic Power

- Decision Trees (Lower Bounds: Sorting & Searching)
 - P, NP, and NP-Complete Problems
-

Prepared by:

Dr. Naveen Kumar K R

(Exam Revision Notes)

© NKR – For private circulation among students.
Intended for easy understanding and quick revision.

Decision Tree Model: Lower Bounds for Sorting & Searching

1. Decision Tree Model

A decision tree is a simple way to look at how algorithms work when they have to compare items to make a choice. Think of it like a flow chart that shows every possible "yes" or "no" path an algorithm can take.

- **Internal Nodes:** These are the circles in the middle of the tree that represent a comparison, like "Is $a < b$?"
- **Edges:** These are the branches coming out of the nodes that show the result of the comparison (usually "yes" or "no").
- **Leaves:** These are the squares at the bottom that show the final answer or outcome.
- **Height (h):** This is the length of the longest path from the top to a leaf. In the world of algorithms, the height represents the **worst-case** performance because it shows the maximum number of comparisons needed.
- **Height Formula:** There is a mathematical rule that says for any binary tree with l leaves, the height must follow the rule $h \geq \lceil \log_2 l \rceil$. This helps us find the "information-theoretic lower bound," which is just a fancy way of saying the absolute minimum number of steps any algorithm must take.

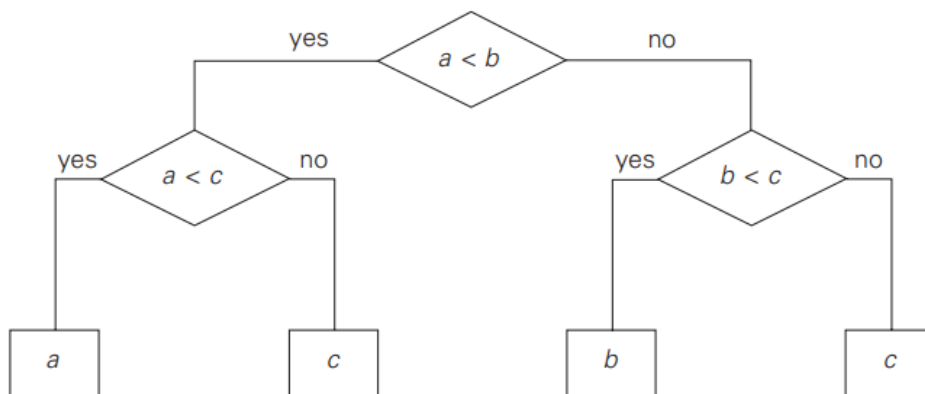


Figure 11.1: Decision tree for finding a minimum of three numbers.

2. Decision Trees for Sorting

When we sort n elements, we are basically trying to figure out which order (permutation) they belong in.

- **Possible Outcomes:** For a list of n items, there are $n!$ (n factorial) possible ways to arrange them. This means our decision tree must have at least $n!$ leaves.
- **Worst-Case Lower Bound:** If we plug $n!$ into our height formula, we get $C_{\text{worst}}(n) \geq \lceil \log_2 n! \rceil$.
- $n \log_2 n$: Using a math trick called Stirling's formula, we find that $\log_2 n!$ is roughly equal to $n \log_2 n$. This tells us that no matter how clever you are, you can't sort things using comparisons faster than this.
- **Examples:**
 - **Selection Sort:** If you draw the tree for sorting 3 elements, you'll see some redundant comparisons where only one outcome is actually possible based on previous steps.
 - **Insertion Sort:** By looking at the average depth of the leaves in the tree for 3 elements, we can find the **average-case** performance. For 3 elements, it takes about 2.66 comparisons on average.
- **Average-Case Lower Bound:** The rule for the average number of comparisons is $C_{\text{avg}}(n) \geq \log_2 n!$, which is also roughly $n \log_2 n$.

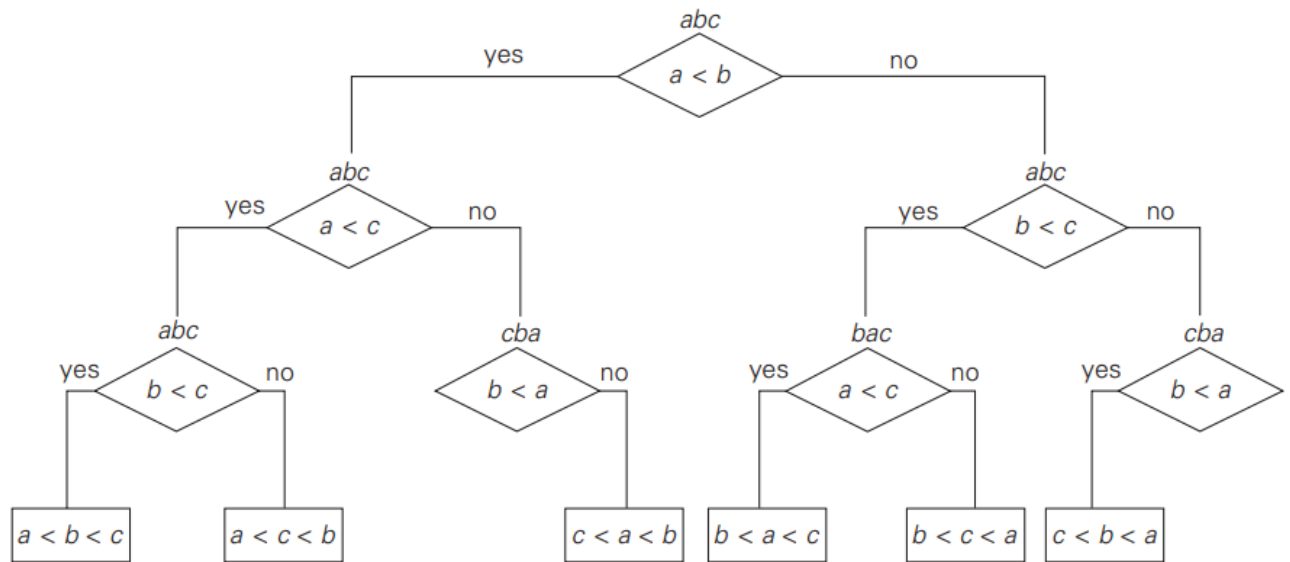


Figure 11.2: Decision tree for three-element selection sort.

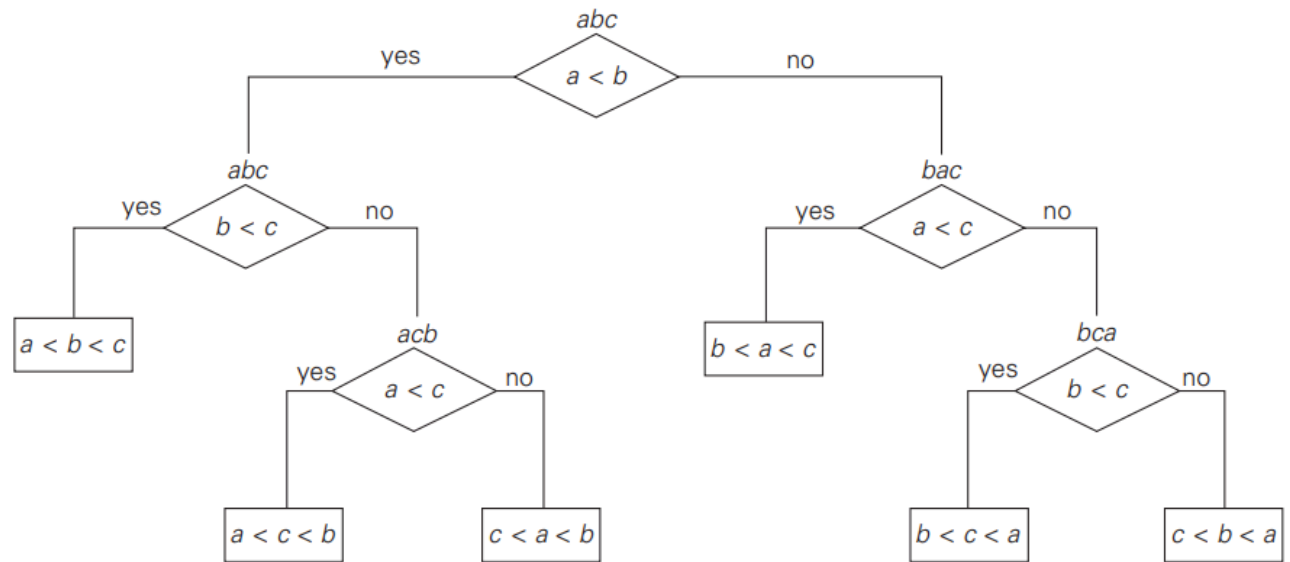


Figure 11.3: Decision tree for three-element insertion sort.

3. Decision Trees for Searching a Sorted Array

When we search for a key K in an array that is already sorted, we use three-way comparisons: $K < A[i]$, $K = A[i]$, or $K > A[i]$.

- **Ternary Decision Tree:** If we show all three outcomes at every step, we get a ternary tree (3 branches per node). This tree has $2n + 1$ leaves (for successful and unsuccessful searches) and gives a lower bound of $\lceil \log_3(2n + 1) \rceil$. However, this bound isn't very strong.
- **Binary Decision Tree:** To get a more accurate lower bound, we use a binary tree where we assume the algorithm stops if it finds a match. The leaves then represent only the $n + 1$ **unsuccessful outcomes** (the spaces between or outside the elements).
- **Optimal Bound:** Applying our height formula to this binary tree gives $C_{\text{worst}}(n) \geq \lceil \log_2(n + 1) \rceil$.
- **Why Binary Search is King:** Since the actual performance of binary search matches this formula exactly, we can prove that binary search is **optimal**—it is literally the best possible way to search a sorted list using comparisons.

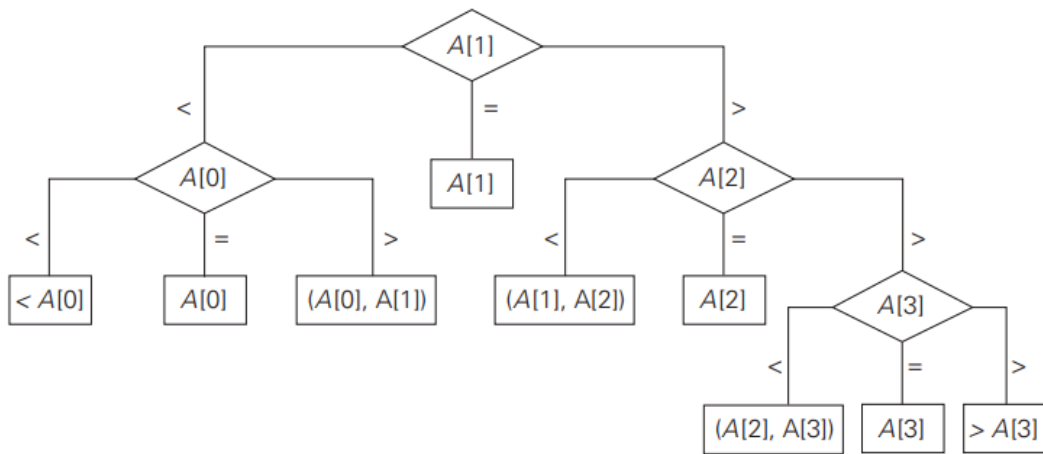


Figure 11.4: Ternary decision tree for binary search in a four-element array.

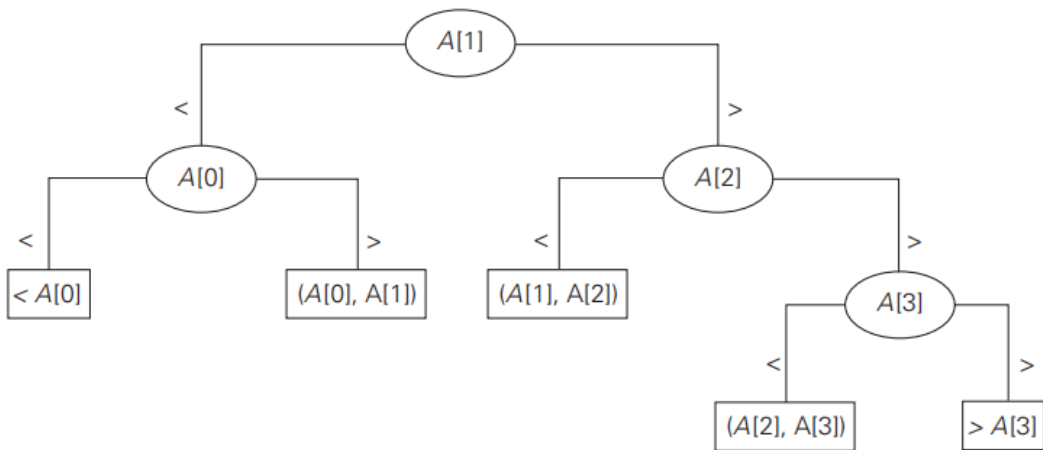


Figure 11.5: Binary decision tree for binary search in a four-element array.

4. Summary Table

Problem	Lower Bound (Worst Case)	Optimal Algorithm
Sorting	$\lceil \log_2 n! \rceil \approx n \log_2 n$	Mergesort
Searching	$\lceil \log_2(n + 1) \rceil$	Binary Search

5. Exam Tips

- **Don't forget the Factorial:** For sorting questions, remember that the number of outcomes is $n!$ because that's how many ways you can shuffle the items.
- **Height = Worst Case:** If an exam asks for the "worst-case comparisons," they are asking for the height of the decision tree.
- **Ternary vs. Binary:** Be careful! Ternary trees are natural for searching, but binary decision trees are used to prove the **tight** lower bound for binary search.
- **Ceilings Matter:** Always use the ceiling symbols $\lceil \rceil$ in your formulas; you can't have half a comparison!.
- **Optimal Algorithms:** Remember that Mergesort is optimal for sorting because its performance matches the $n \log n$ lower bound.

Computational Complexity – P, NP, NP-Complete

1. Polynomial Time and Tractability

- **Polynomial-Time Algorithm:** An algorithm is said to solve a problem in polynomial time if its worst-case time efficiency belongs to $O(p(n))$, where $p(n)$ is a polynomial function of the input size n .
- **Tractable Problems:** These are problems that can be solved in polynomial time. They are considered "easy" or manageable for computers even as input sizes grow.
- **Intractable Problems:** These are problems that cannot be solved in polynomial time. They are considered "hard" because as the input size increases, the time required to solve them grows so fast that it becomes impossible for any computer to handle them.
- **Choosing Polynomial Time as the Boundary:** Computer scientists use polynomial time as the line between tractable and intractable for four main reasons:
 1. **Exponential Growth:** Non-polynomial functions (like exponential growth) become astronomically large very quickly, making algorithms impractical.
 2. **Low Typical Degrees:** Most useful polynomial-time algorithms have a low degree, usually three or less.
 3. **Closure Properties:** The sum or composition of two polynomials is always another polynomial, which is convenient for mathematical theory.
 4. **Model Consistency:** A problem's tractability generally stays the same across different types of computers and input-encoding schemes.

2. Class P

- **Formal Definition:** Class P is the set of all **decision problems** that can be solved in polynomial time by a **deterministic** algorithm.
- **Use of Decision Problems:** A decision problem is one that has a simple "yes" or "no" answer. The theory focuses on these because:
 1. It excludes problems where the output itself is exponentially large (like listing all subsets of a set).

2. Many complex optimization problems (finding the best solution) can be reduced to a series of simpler decision problems.

- **Examples of Problems in P:** The following problems are known to have polynomial-time algorithms:
 - Sorting a list and searching for a key.
 - Computing the product and the greatest common divisor (gcd) of two integers.
 - Checking if a graph is connected or has cycles (acyclicity).
 - Finding a Minimum Spanning Tree (MST) or the shortest paths in a weighted graph.

3. Class NP and Nondeterministic Algorithms

- **Nondeterministic Algorithm:** This is a two-stage procedure for solving a decision problem:
 1. **Guessing Stage:** An arbitrary string is generated as a "candidate solution".
 2. **Verification Stage:** A deterministic algorithm checks the candidate solution to see if it is valid.
- **Nondeterministic Polynomial (NP) Algorithm:** An algorithm where the **verification stage** runs in polynomial time.
- **Class NP:** The set of all decision problems that can be solved by a nondeterministic polynomial algorithm.
- **Relationship ($P \subseteq NP$):** Every problem in P is also in NP. This is because if you can solve a problem in polynomial time, you can also verify a guessed solution in polynomial time by just ignoring the guess and solving it yourself.
- **Examples of Problems in NP:**
 - Hamiltonian circuit problem (finding a cycle visiting every vertex once).
 - The partition problem and the decision versions of the Traveling Salesman Problem (TSP), knapsack, and graph coloring.
- **Non-NP Example:** The **halting problem** (determining if a program will stop or run forever) is an example of a decision problem that is **not** in NP.

4. NP-Complete Problems and Polynomial Reduction

- **Polynomial Reduction:** A decision problem D_1 is polynomially reducible to D_2 if there is a function that transforms instances of D_1 into instances of D_2 in polynomial time.
 - **Implication:** If D_2 can be solved in polynomial time ($D_2 \in P$), then D_1 must also be solvable in polynomial time ($D_1 \in P$).
- **NP-Completeness (Informal):** An NP-complete problem is a problem in NP that is "as hard as" any other problem in NP.
- **NP-Completeness (Formal):** A decision problem D is NP-complete if:
 1. It belongs to class NP.
 2. Every other problem in NP is polynomially reducible to it.
- **Proving NP-Completeness:** To prove a new problem is NP-complete, you must:
 1. Show it belongs to class NP.
 2. Transform a **known** NP-complete problem into the new problem in polynomial time.
- **Example from Text:** The Hamiltonian circuit problem is polynomially reducible to the decision version of the Traveling Salesman Problem (TSP).

5. Important NP-Complete and Hard Problems

The following are well-known NP-complete problems (or their decision versions):

- **CNF-Satisfiability Problem:** Determining if a boolean expression in conjunctive normal form can be true; this was the **first** known NP-complete problem (Cook-Levin theorem).
- **Hamiltonian Circuit Problem:** Determining if a path exists that visits every vertex once and returns to the start.
- **Traveling Salesman Problem (Decision):** Checking if a tour through n cities exists with a length not exceeding a given integer m .
- **Knapsack Problem:** Finding a subset of items with given weights and values that fits in a bag of a specific capacity.
- **Partition Problem:** Determining if a set of integers can be split into two groups with the same sum.
- **Bin-Packing Problem:** Fitting items into the minimum number of bins of size 1.

- **Graph-Coloring Problem:** Finding the smallest number of colors needed so that no two adjacent vertices share the same color.
- **Integer Linear Programming Problem:** Maximizing or minimizing a linear function with integer-valued variables under constraints.

6. The P versus NP Question

- **The Statement:** The most important open question in computer science is whether $P = NP$ or $P \neq NP$.
- **Significance:** If $P = NP$, it would mean every NP-complete problem could be solved by a polynomial-time algorithm.
- **Current Belief:** Most computer scientists believe $P \neq NP$. The main reason is that despite decades of searching, no one has ever found a polynomial-time algorithm for any NP-complete problem.
- **Practical Implications:** If you encounter a problem known to be NP-complete, you should stop trying to find an exact polynomial-time algorithm. Instead, focus on alternatives like approximation techniques or heuristics.

7. Comparison of P, NP, and NP-Complete

Feature	Class P	Class NP	NP-Complete
Solvable in Poly-time?	Yes (Deterministic)	Not proven for all	No known algorithm
Verifiable in Poly-time?	Yes	Yes (Nondeterministic)	Yes
Relationship	$P \subseteq NP$	Parent class of P	Hardest problems in NP

Summary of Relationship:

- P is a subset of NP .
- NP-complete problems are a subset of NP .
- If **any** NP-complete problem is ever solved in polynomial time, then $P = NP$.

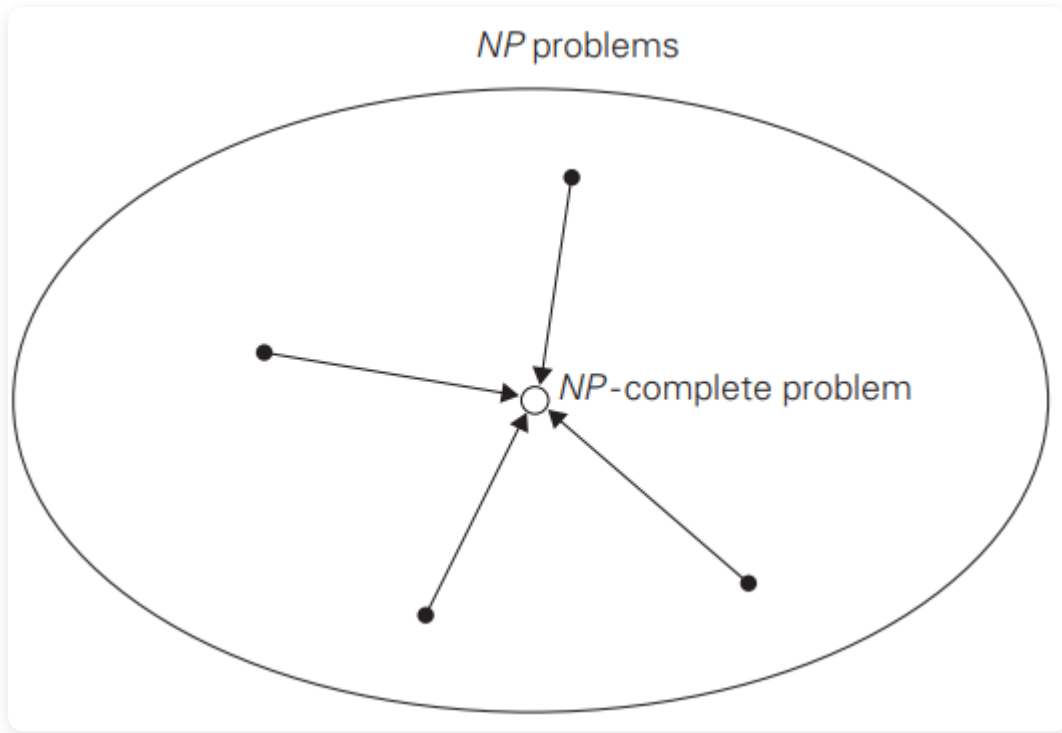


Figure 11.6: Notion of an NP-complete problem. Polynomial-time reductions of NP problems to an NP-complete problem are shown by arrows.

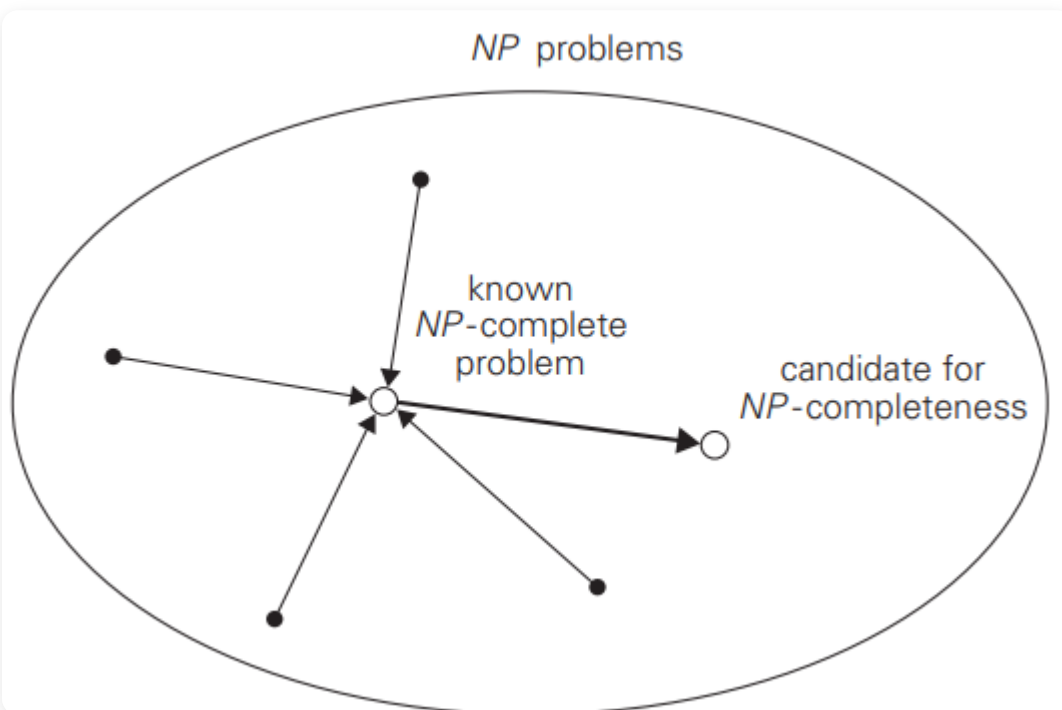


Figure 11.7: Proving NP-completeness by reduction.

8. Examination Tips and Common Pitfalls

- **Decision Focus:** Always state that P and NP are classes of **decision problems** (yes/no).
- **NP Definition:** Do not say NP means "non-polynomial." It stands for "**nondeterministic polynomial**".
- **Tractable vs. Intractable:** Use "tractable" for problems in P and "intractable" for those proven to be beyond P.
- **Reduction Precision:** In proofs, you must show the transformation is computable in **polynomial time** and that it correctly maps "yes" and "no" instances.
- **The Halting Problem:** Use this as an example of an **undecidable** problem, meaning no algorithm exists to solve it at all.
- **First Problem:** Always mention the **CNF-satisfiability problem** and the **Cook-Levin theorem** as the first proof of NP-completeness.

Summary

- **Tractable** problems (Class P) can be solved efficiently.
 - **NP** problems can be **verified** efficiently if you are given a guessed solution.
 - **NP-complete** problems are the toughest in NP; solving one solves them all.
 - $P \neq NP$ is the common belief, meaning checking a solution is easier than finding one.
-